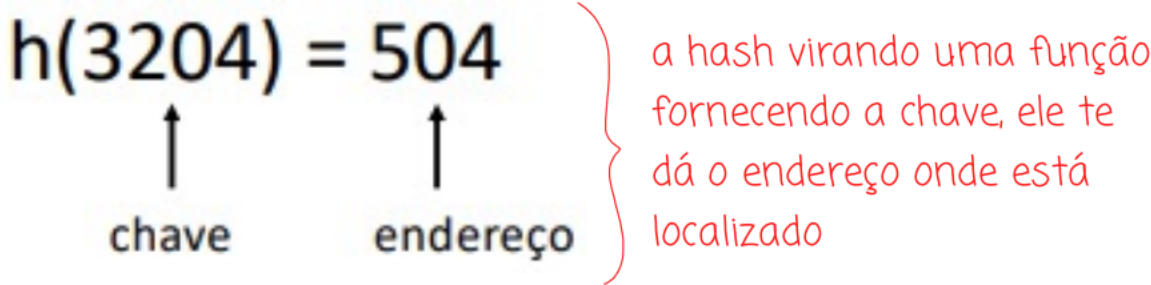


NOTEBOOK

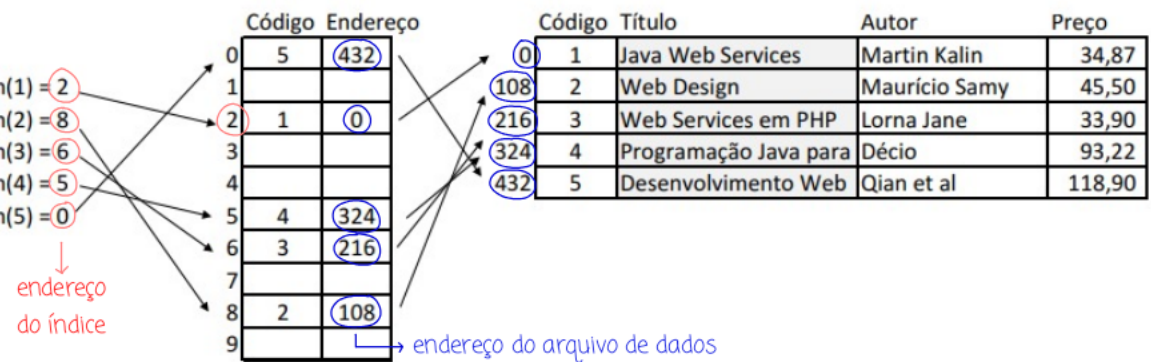
Tabelas hash

as tabelas hash em disco também podem ser usadas como índices ao invés das arvores.

o acesso dela é direto, sendo um custo de $O(1)$.



Obs: a função hash ela retorna o endereço do índice, não do arquivo.



endereçamento aberto: usa outras posições vazias dentro da própria tabela hash.

Encadeamento externo: usa uma área extra, além da tabela hash, como uma área de extensão. (hash direta com área de reserva).

Encadeamento externo: usa uma área externa fora da tabela. (hash indireta com lista flexível).

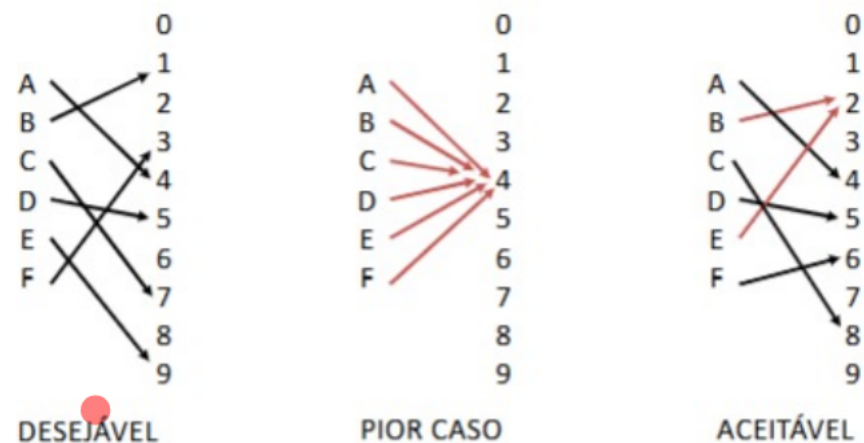
Sondagem linear : as próximas posições são sondadas (circularmente) até que uma posição esteja disponível. E caso a posição original for apagada e tivermos outro elemento com a mesma chave em outra posição, marcamos uma lápide no endereço original, para que saiba que existe outro arquivo com a mesma chave na tabela

Sondagem quadrática: a distância até a próxima posição é determinada pelo quadrado da tentativa.

Regra: $h(k, i) = [h(k) + i^2] \bmod n$

Duplo hash: a distância até a próxima posição é determinada por uma segunda função, ou seja ele soma a posição de conflito e a nova posição que a hash forneceu e insere nela.

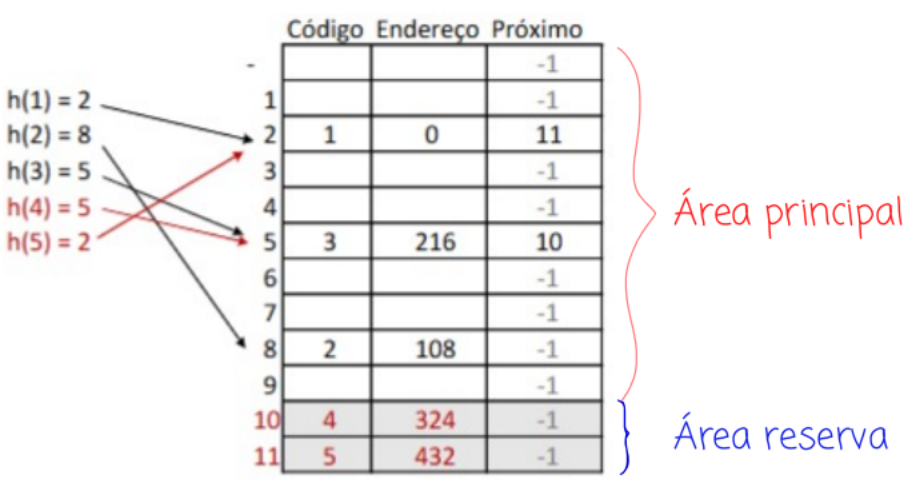
Colisões



Encadeamento

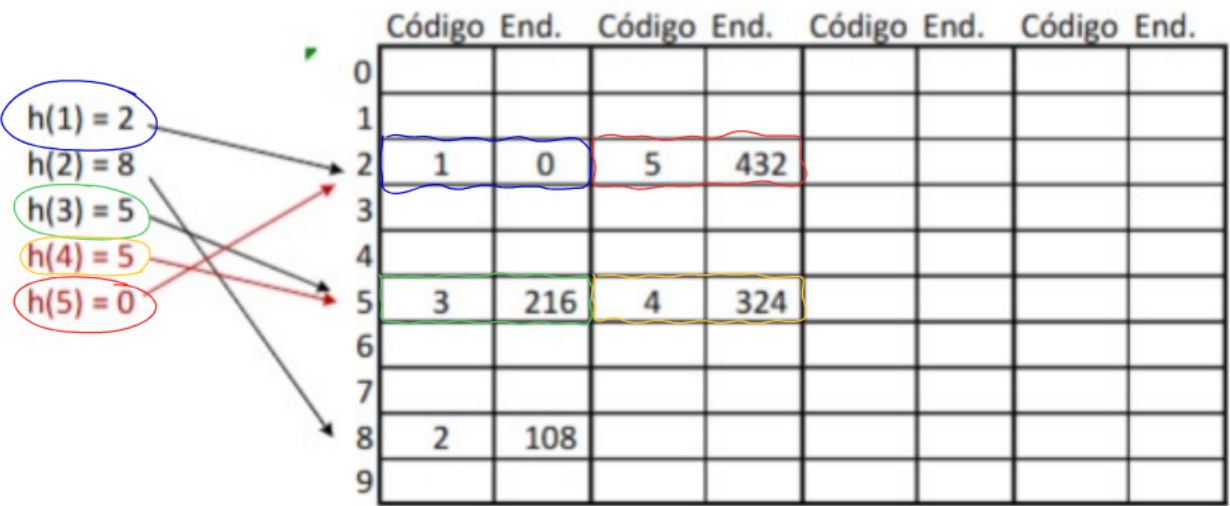
Encadeamento interno

Os registros coloridos são armazenados em uma área extra (geralmente fora da área principal).



Cestos (buckets)

cestos são posições da tabela hash que guardam múltiplos elementos em uma lista.



E quando o cesto estiver cheio?

usaríamos as mesmas técnicas anteriores, porém ao invés de apontar para um índice, apontaríamos para outro cesto.

Encadeamento externo

Todos os registros são armazenados em uma lista encadeada (outro arquivo).

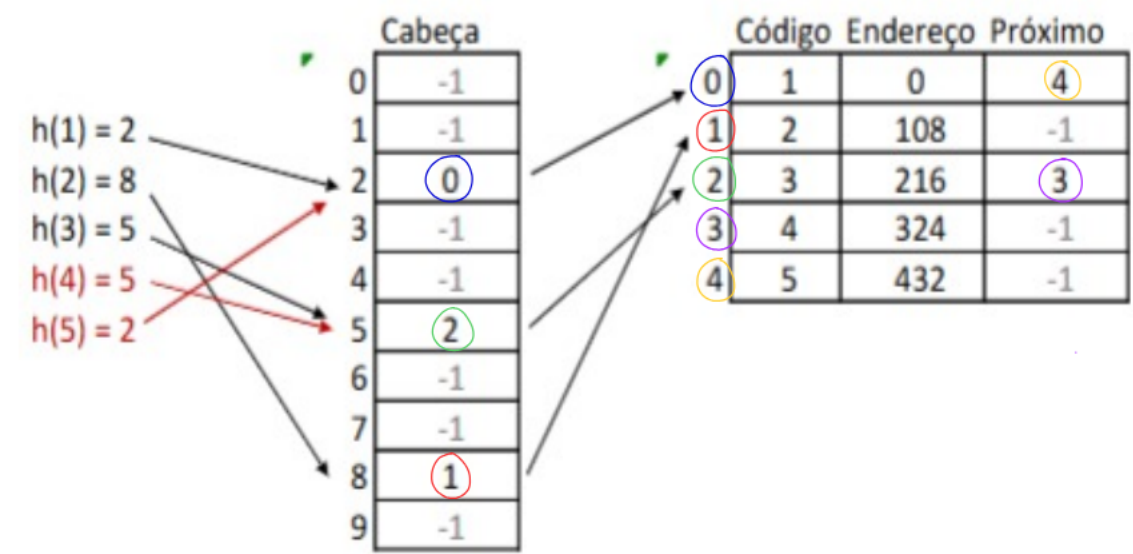


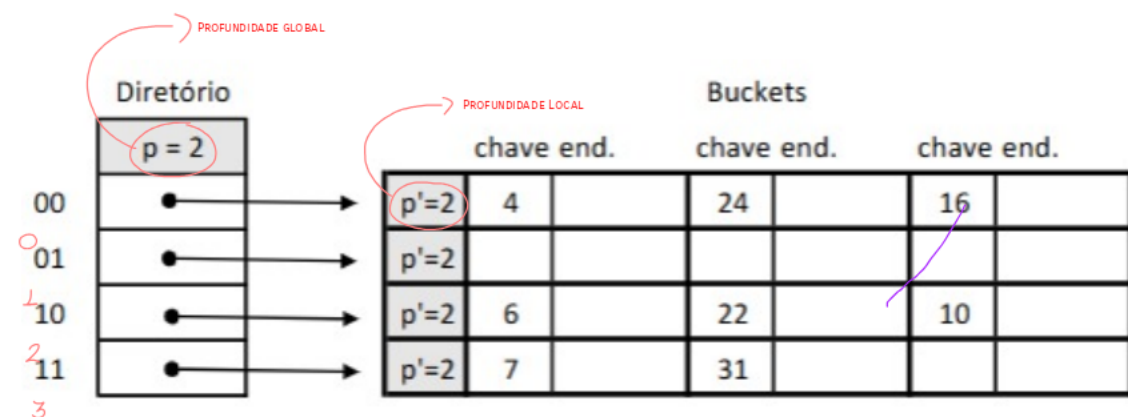
Tabela hash dinâmica

quando o arquivo de dados cresce ou diminui com frequência.

uma tabela hash estática, para crescer precisa reposicionar todos os registros.

Já a tabela hash extensível é uma tabela hash em que apenas alguns registros são afetados, os registros dentro do buckets.

Exemplo de implementação



aqui temos 4 endereços para posicionar os elementos, usando por exemplo um $\text{mod}(\text{tamanho do diretório})$.

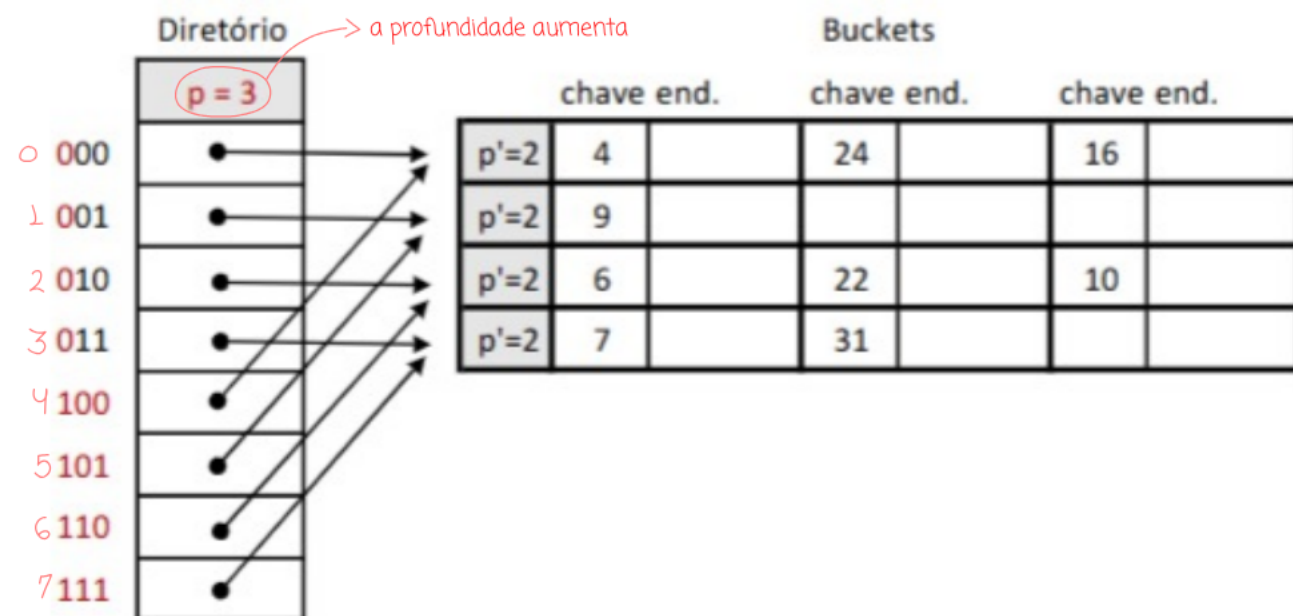
por exemplo : $4 \bmod (4) = 0$, está na posição correta.

$6 \bmod (4) = 2$, está na posição correta

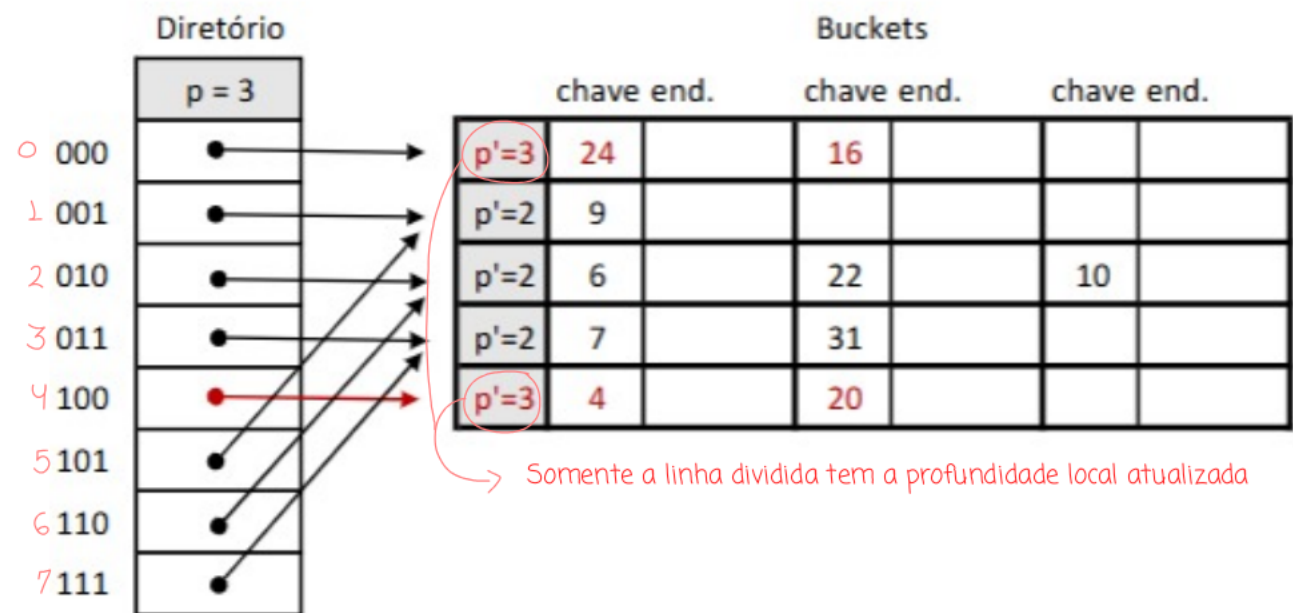
E se quisermos adicionar um elemento em um bucket cheio?

Adicionar o 20, que $20 \bmod (4)$ é 0.

quando um bucket está cheio, nós **dobramos o diretório** para que mais posições possam ser inseridas.



os elementos novos apontam ainda para os mesmos endereços, **enquanto não houver um "estouro" na linha**



Como inserimos o 20, houve um estouro nessa linha, então separamos as linhas e fizemos o mod dos elementos presente nela. **lembrando que o mod passou a ser 8 agora.**

Vantagens do hash extensível

- O diretório cresce, sempre dissemos reposicionar TODOS os registros.
- O índice (lista de buckets) cresce de acordo com a necessidade.
- Como não há encadeamento, não a hanperda de eficiência.